MS2 Summary: skyjam, Forecasting GNSS Interference over European Airspace

Yazdan Musa (github.com/Zayden16) · MLOps HS26 · MS2 Feature Pipeline · `https://github.com/Zayden16/hslu-mlops-hs-26`

Primer (B1). Aircraft broadcast their position over *ADS-B* together with a *NIC* value (0–11) saying how much they trust their own GPS fix. NIC below 7 means the fix is worse than 0.2 NM (370 m), which is what jamming does. We only use aircraft above *FL200* (20 000 ft), because small low aircraft report bad NIC everywhere. The map is cut into H3 hexagons of about 180 km edge. One *cell-hour* is one hexagon in one UTC hour, e.g. "cell over the Gulf of Finland, 14:00: 23 aircraft, 14 with NIC < 7, share 0.61, so *degraded*". The task: is that cell degraded 6 hours later? The 18 *sampling discs* are the circles (about 250 NM radius) the API is queried with.

1 What I achieved in this milestone

- **The feature pipeline now runs on its own.** A second Railway service, `features`, runs at minute 20 of every hour (`railway.json`, cron `20 * * * *`), 20 minutes after the Go ingestor's sweep. It reads raw `observations`, builds cell-hours, lags, rolling means, neighbour features and the 6 h target, and swaps the result into the Postgres table `cell_hourly_features`. Every run writes one row to `feature_runs` with row count, slot range, positive rate and `feature_version`. In MS1 only capture was automated. Features were computed by hand.
- **Backfill is a full rebuild.** The feed has no history endpoint, so a lost hour cannot be downloaded again. Instead, raw observations are stored unfiltered, and each run recomputes *all* history (3.3 M cruise rows to 54 686 labelled rows, about 40 s). A new feature or threshold therefore applies to the whole dataset at once. I chose this over incremental updates because it is simpler and cannot go stale.
- **Neighbour-cell features (E1)** in `features/compute.py`: share and state of the 6 adjacent cells, their lags, and a 3 h change of the neighbour share.
- **Leakage tests on the real path (B2)**: `tests/test_pipeline_leakage.py` starts from raw aircraft rows and runs the production functions. 43 tests in total, also against a real Postgres in CI.
- **What it cost me.** In October the ingestor lost 78 hours, nearly every afternoon. adsb.lol has a quota of 500 requests per day and 30-minute sweeps used 864. Sweeps are now hourly (`168b31a`, analysis in `docs/ingestion-gaps-2026-10.md`). Completeness since 17.09: **1 138 of 1 254 hours (90.7 %)**, November so far 181/181. I was also ill for most of the final week, so model training has not started.
- **Look first at:** `src/skyjam/features/compute.py`, `tests/test_pipeline_leakage.py`, `write_features` in `src/skyjam/c` `railway.json`, and the README pipeline table.

2 Response to MS1 reviews

E1 / A3 · Beat persistence where it matters, neighbour features. [**Partly**] Neighbour features are built and tested (`adacf4e`). The evaluation on transitions is set up and will be reported in MS3. What the data already shows: 7.7 % of rows are transitions (4 188, of which 1 974 onsets). Persistence has PR-AUC 0.39 overall but, by definition, *zero* recall on transitions. Among currently clean cells, the onset rate is 4.0 %, but 18.8 % if at least one neighbour is degraded. As a single score for onsets, the maximum neighbour share reaches average precision 0.22 against 0.19 for the cell's own share (base rate 0.04). So the reviewer's point holds in the data. MS3 will report PR-AUC and recall overall and on transitions only, against both baselines.

E2 · Remove `ilias-dump/` from the public repo. [**Changed**] The folder is removed from the full git history with `git filter-repo` and the repository was force-pushed. It is in `.gitignore` (`4c0ffff`) and `.dockerignore`.

A1 · Rerouting and the five-aircraft floor may hide jammed cells. [**Partly**] Measured on all 103 804 cell-hours: 30.2 % fall below 5 aircraft. The drop rate differs a lot by disc: 6 % over Switzerland, 61 % over the Gulf of Finland, 73 % over the Caucasus, 99 % over northern Finland. Across discs it correlates with the degraded rate ($r = 0.43$), and dropped cell-hours have a degraded share ≥ 30 % twice as often (22 % vs 10 %). The risk is therefore real. Per cell the link is weak (Spearman 0.09), so it is mostly regional. The floor stays, because one bad airframe would otherwise make a 100 % outage. The neighbour features partly recover the signal, since a jammed area usually still has some cells above the floor. For MS3 I will add the count of thin neighbour cells as a feature and report the metrics per region.

B2 · Leakage tests on the real pipeline. [**Changed**] `tests/test_pipeline_leakage.py` builds a small known example from raw rows through `assign_cells` → `aggregate_cells` → `build_features` and checks: a 1 h lag after a missing hour is empty (not the value from 2 h before), the 6 h rolling mean excludes the current hour, changing every hour after T leaves all features at T identical, the target is the state at $T + 6$, thin cells are dropped, not treated as clean, and the same three checks for neighbour features.

E3 · Where MLflow runs. [**Not yet**] Decided, not deployed: an MLflow server as a third Railway service, backend store in the existing Postgres (separate `mlflow` database), artifacts on a Railway volume mounted at `/mlflow`. Inference loads `models:/skyjam@production` from that server. This is part of MS3.

E4 · Data volume for MS3. [**Changed**] My MS1 estimate was too high. Actual: 54 686 labelled cell-hours over 51 days (about 1 050 per day, not 1 900, because of the October gaps and the traffic floor), 95 cells, 9.2 % positives. With a 70/15/15 chronological split the test set (29.10–08.11) has 8 203 rows, 512 positives and 304 onsets. Requiring a full week of history for the 168 h lag would keep only 75.7 % of rows. So the 168 h lag stays as a missing value, which LightGBM handles, rather than a filter. By 03.12 the test set will roughly double.

A2 · Spoofing. [**Changed (here)**] A spoofed aircraft reports a false position, often with normal NIC. It can then be put in the wrong cell and does not count as degraded, so spoofing mostly causes *missed* labels, not false ones. skyjam predicts loss of integrity, and its label cannot see undetected spoofing. This is now stated as a limit.

B1 · Readability. [**Changed**] Primer and a cell-hour example at the top of this page. I also wrote this summary in plainer language, as the editors asked.

E5 · Full name. [**Changed**] Full name in the title block of this document.